



BITS Pilani

Introduction to AI Systems

Dr. Minati Rath
mrath@wilp.bits-Pilani.ac.in



<SS ZG662, Introduction to AI Systems>

Session 1

These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.



- Structured and Unstructured Data,
- Data Sources,
- Data Quality,
- Data Governance

Structured vs Unstructured: A Legacy Classification



Traditional: Structured, Semi-structured, Unstructured

Modern AI perspective:

Raw data



Representation



Semantic structure



Machine-consumable representation

LLMs, embeddings and multimodal models fundamentally change the economics of unstructured data.

“My payment went through twice, but I only made one purchase. Please refund the extra charge.”

A traditional system might extract:

Customer ID: 18392

Issue type: Duplicate payment

Transaction: T82931

Amount: ₹4,500

Status: Refund requested (This requires predefined schemas, extraction rules, NLP pipelines, and significant engineering effort.)

Issue - *How do we convert every piece of unstructured data into a rigid relational schema*

Structured vs Unstructured: A Legacy Classification



Modern AI perspective

Raw: "My payment went through twice..."

Semantic representation:

Customer → experienced → duplicate payment

Payment → amount → ₹4,500

Customer → requested → refund

Issue → category → payment duplication

Embedding: [0.12, -0.37, 0.81, ...]

Now the enterprise can ask questions that were previously difficult to answer: "*Find all customers whose payment problem is similar to this complaint.*"

~~The system can use embeddings to find semantically similar cases—even if they use completely different words: "I was charged twice" "Money deducted two times" "Duplicate debit on my account" "Same transaction appears twice" They don't need to have the same keywords.~~

The key change is not simply that AI can understand unstructured data.

It is the cost of *converting unstructured data into useful machine-consumable representations* has fallen dramatically.

The Hidden Structure Inside Unstructured Data



Example: **Contract**

Raw: 200-page PDF

Extract:

Parties

Dates

Obligations

Payment terms

Penalties

Renewal clauses

Jurisdiction

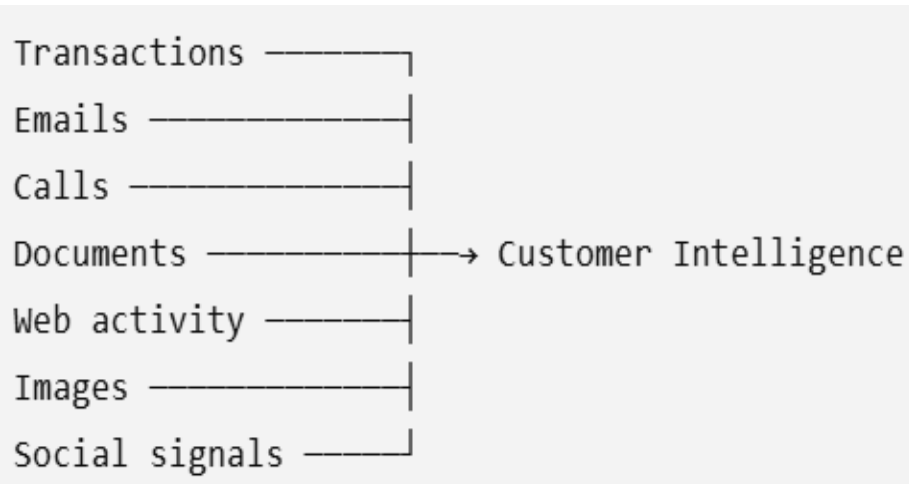
Then: Document → entities → relationships → knowledge

So: *Unstructured data often contains highly structured business information.*

Enterprise already has data management. The question is what additional capabilities are required when data becomes the fuel for ML and AI systems.

The Enterprise Multimodal Data Stack

Example: Customer



- *Multimodal AI*
- *Entity resolution*
- *Context fusion*
- *Cross-modal retrieval*

Traditional objective: reliable reporting, transaction processing, regulatory reporting, BI and analytics, data sharing.

New objective: continuous machine consumption, automated decisions, predictive systems, generative AI, AI agents.

Data adequate for reporting is not necessarily adequate for machine learning.

Enterprise Data Management Was Designed for a Different World

A table looks structured. But what does: customer_status = 3 mean?

Possibilities: Active, Suspended, Dormant

Therefore: **Schema is not semantics.**

We need **Business definitions Metadata, Data dictionaries, Ontologies.**

Temporal Data Is Different

A critical advanced issue.

Customer credit score

January-720, March-680, June -640

Which value was available **at the time the prediction was made?**

Which value was available at the time the prediction was made?

March 15

Prediction made

Credit score = 680



Customer eventually defaults



June - Credit score updated = 640

In August, a data engineer creates the training dataset by joining tables:

Prediction table

Customer | Prediction_Date

A | March 15

JOIN

Customer table

Customer | Current_Credit_Score

A | 640

Temporal Data Is Different



The resulting training data becomes:

Customer | Prediction_Date | Credit_Score

A | March 15 | 640 ← WRONG

The database query has no idea that **640 was not available on March 15**. It simply retrieves the customer's current/latest value.

This is the key problem

A normal database query might effectively ask: "What is Customer A's credit score?"

An ML training query needs to ask: "What was Customer A's credit score as it was known on March 15?"

point-in-time correctness is largely about preserving and using temporal information correctly when reconstructing historical ML datasets.

AI/ML-oriented data management, **"current value"** is often not enough. You need to know **what the value was, when it became valid, and when it became available to the prediction system.**

Event Data vs State Data



State - Customer balance = ₹4.2 lakh

Event - Customer transferred ₹50,000 at 14:37:21

Events allow reconstruction of state.

But state does not necessarily allow reconstruction of history.

This distinction affects: Fraud, Recommendations, Forecasting, Causal analysis, ML training

Data Leakage: The Silent Killer

Example: Predict: Will a customer default?

Feature: “Collections action initiated”

Looks predictive. But collections action occurred **after** the customer was already identified as risky.

Model gets artificially high performance.

Training data accidentally contains future information.

The Real Data Question is Not: Structured or unstructured?

Instead: Can we represent the information in a form that preserves the semantics required for the decision?

ENTERPRISE DATA SOURCES



~~Data sources reveal how an organization operates.~~

Examples: ERP → transactions, CRM → relationships, HR → people
MES → manufacturing, Logs → digital behavior, Sensors → physical world

Data architecture is often a mirror of organizational architecture.

Source-of-Truth Conflicts *Example: Customer address:* CRM: Bangalore,
ERP: Bengaluru, Shipping: Whitefield, KYC: Mysore

Which is correct? Depends on the business decision.

There may be no universal “truth.”

*Production databases
are not automatically
training datasets.*

**Operational
systems optimize
for:** Transaction
integrity, Business
workflows,
Availability,
Accuracy

ML requires:
Historical context
Feature computation
Labels, Bias, Temporal
consistency, Leakage,
Reproducibility,
Distribution stability,
Point-in-time correctness

Data Integration Is Not
Data Unification.
Integration can
technically join them.
Semantic unification
determines whether
~~they should be joined.~~



Vendor says:

“Our dataset has 99% accuracy.”

Ask:

Accuracy measured against what?

Sampling method?

Geography?

Time period?

Population?

Missing values?

Collection methodology?

Version?

Provenance is part of data quality.

Third-Party Data Risk

External datasets create:

- Licensing risk
- Dependency risk
- Availability risk
- Version risk
- Bias risk
- Vendor lock-in

What happens to your model if the supplier changes its methodology tomorrow?

Human-Generated Data



Examples:

Call-center notes

Technician comments

Sales notes

Medical narratives

Incident reports

Problem:

Humans don't generate data according to database schemas.

They generate **language**.

This creates:

Ambiguity , Subjectivity

Missing context, Bias

Inconsistent terminology

Machine-Generated Data

IoT/log data can create:

millions of events per second

The challenge changes from:

“How do we collect data?”

to:

“What data should we retain?”

- Sampling
- Aggregation
- Compression
- Edge processing
- Retention

Synthetic Data: Solution or Illusion?



Potential benefits:

Rare-event generation

Privacy, Simulation, Testing

Risks:

Distribution mismatch

Mode collapse, Hidden bias

Unrealistic correlations

Does synthetic data preserve the relationships that matter for the decision?

Data Acquisition Has an Economic Cost

Every dataset has:

**Acquisition cost + Storage cost +
Processing cost + Governance cost +
Opportunity cost**

More data $\neq$ more value

Data Quality Is Decision-Dependent

Example: GPS accuracy of ± 10 meters:

Fine for city-level analytics

Bad for autonomous navigation

Data quality cannot be defined independently of the use case.

Labels Are Business Judgments



Example: “Fraud” Who decides?

Bank investigator

Rule engine

Customer complaint

Chargeback

Law enforcement

Each may produce a different label.

A label is not necessarily ground truth.

The Label-Delay Problem

Suppose we want to predict:

Customer churn. But churn becomes observable only **90 days later**.

Prediction time $\neq$ label availability time

This affects: Training, Evaluation, Retraining, Experimentation

Data Quality vs Data Observability

Traditional: Check data periodically.

Modern: Continuously observe the data pipeline.

Monitor: Volume, Freshness,

Distribution, Schema, Missingness

Cardinality, Statistical properties

Data $\rightarrow$ metrics $\rightarrow$ alerts $\rightarrow$ diagnosis

Data Contracts



Producer: income = 50000 ₹

Changes to: income = 50000 USD

Data Quality Gates

Ingest → Schema Validation → Quality Checks → Business-Rule Checks → Statistical Checks → Approved Dataset → ML Pipeline

Should bad data fail the pipeline—or merely generate an alert?

It is a **business-risk decision**, not merely an engineering decision.

Data Quality Has a Financial Value

1% Incorrect Records → 20M Transactions → Business Impact

Cost of Poor Data Quality Vs Cost of Improving Data Quality

This reframes data quality from: **IT hygiene** to: **Investment decision**

The Data Quality Paradox

More validation can mean: Better quality, Slower pipelines, Higher infrastructure cost, More engineering effort

100% data quality may be economically irrational. Optimize for: risk-adjusted quality.

DATA GOVERNANCE FOR AI



Traditional governance asks: Who can access the data?

AI governance must additionally ask:
What decisions can the system make using that data?

Data Governance $\neq$ Data Security

Security: Can you access it?

Governance: Should you access it?

Should this data influence this decision?

Purpose Limitation

Data collected for: Customer support is later proposed for: Employee performance prediction. Technically possible.

But: Is it appropriate?

Governance becomes a *business ethics* + *risk* + *legal* problem.

Data Lineage for AI

Source $\rightarrow$ Transformation
 $\rightarrow$ Feature $\rightarrow$ Training
Dataset $\rightarrow$ Model Version
 $\rightarrow$ Prediction $\rightarrow$ Business Decision

If a regulator asks:

“Why did this system make this decision?”

We need to trace the chain.

RACI Framework - responsibility-assignment framework

Letter	Meaning	Simple interpretation
R	Responsible	Does the work
A	Accountable	Owens the final outcome / decision
C	Consulted	Provides expertise/input before the decision
I	Informed	Kept informed after/because of the decision

Activity	Business	Data	IT	ML	Risk
Definition	A	R	C	C	C
Quality	A	R	R	C	C
Access	C	C	R	C	A
Model use	A	C	C	R	A
Risk	A	C	C	C	R

Responsible ≠ Accountable

*IT may be **Responsible** for implementing access controls, but Risk may be **Accountable** for whether the access policy meets regulatory requirements*

Existing capability → Additional AI requirement.

Data quality → Fitness for AI use.

Data lineage → Training-data lineage.

Metadata → AI-oriented metadata.

Data versioning → Reproducible datasets.

Historical data → Point-in-time data.

Access control → Purpose-based AI access.

Data contracts → ML-aware contracts.

Monitoring → Drift and distribution monitoring.



Centralized

Advantages:

- Consistency
- Control
- Standards

Problems:

- Slow
- Bureaucratic
- Central bottleneck

Federated

Advantages:

- Business ownership
- Faster decisions
- Domain expertise

Problems:

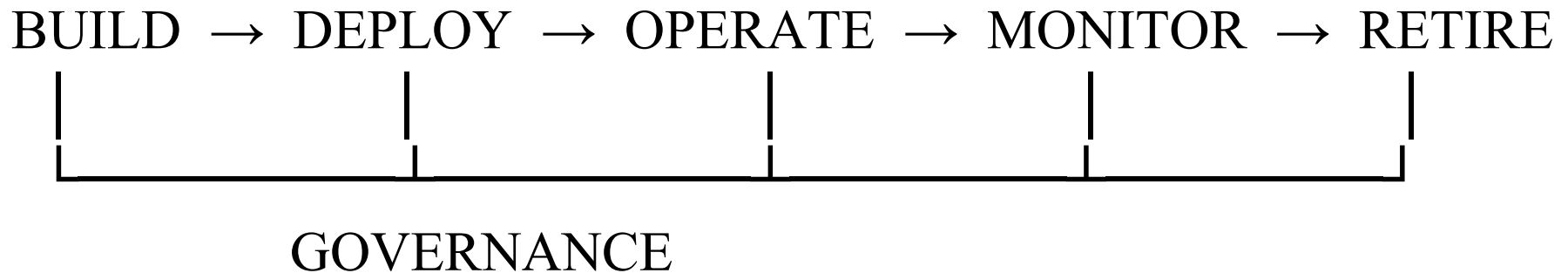
- Inconsistency
- Duplication
- Governance gaps

Federated execution + centralized standards.

Bad architecture:



Better:



*Governance becomes part of the **engineering lifecycle**.*



Five levels: **Level 1 — Fragmented** - Data silos.

Level 2 — Centralized - Warehouse/lake.

Level 3 — Governed - Ownership, quality, lineage.

Level 4 — Data Products - Reusable, discoverable, reliable datasets.

Level 5 — Intelligent Data Platform – governance, observability, lineage and automated controls.

Data continuously feeds:

ML, GenAI, Automation, Decision systems

Where is your organization today?

And more importantly: *What prevents moving to the next level?*

Don't Build an AI Data Platform in Isolation



Bad approach: Enterprise Data Platform plus separate AI Data Platform.

Better approach: extend the enterprise data platform so that traditional consumers and AI consumers use governed data products.

Principle: extend enterprise data management rather than creating an isolated AI data island.



Operational systems → ETL → Data Warehouse → BI.

Typical optimization: reporting, aggregation and historical analysis.

Question: what additional architectural requirements appear when data is consumed continuously by machines?



Operational systems, CRM, ERP, applications, IoT and external sources feed an integration layer and enterprise data platform. The platform may include warehouse, lakehouse and event stores. Consumers include BI, analytics and AI. Implementation question: which workloads belong where?



Should we use streaming? Vs How quickly does the business decision become stale?

Annual planning may use batch; daily pricing may use batch.

Recommendations may be near-real-time; fraud and machine safety may require real-time.

Implement a decision-latency SLA.

Data Architecture Must Support Multiple AI Consumers

innovate

achieve

lead

One enterprise dataset may serve BI, ML, GenAI/RAG, agents, search and automation.

Do not build one dataset per project.

Build reusable governed data products with clear interfaces and controls.



Traditional: `customer_master_final_v8`.

Data product: Customer 360 with definition, owner, schema, quality SLA, freshness SLA, lineage, access policy, version and approved use cases.

Treat important datasets as managed products rather than anonymous tables.



Example: Customer 360.

Owner: Chief Customer Officer.

Freshness: less than 1 hour.

Completeness: greater than 99.5%.

History: 7 years.

PII: Restricted.

Approved uses: customer analytics and churn modelling.

Restricted uses: employee profiling and external sharing.



Freshness SLA: data available within 15 minutes.

Quality SLA: completeness greater than 99%.

Availability SLA: 99.9%.

Semantic SLA: business definition cannot change without versioning.

Change SLA: breaking schema changes require notification.



Overall quality can hide failures in important populations.

Example: overall 99%, Region A 99%, Region B 98%, Region C 72%.

Implement quality monitoring by geography, product, customer segment, time and channel.



Do not only calculate $\text{NULL} = 5\%$.

Investigate why values are missing.

Analyze missingness by source, segment, time and business process.

Implement missingness profiling and alerts.

Key insight: missingness can represent business behavior.



Example: `customer_id = STRING`; `amount = DECIMAL`;
`transaction_time = TIMESTAMP`.

Useful, but insufficient for AI.

An AI consumer also needs semantic, temporal, quality, statistical, privacy and approved-use expectations.



Semantic: what does the field mean?

Temporal: when was the value known?

Quality: what threshold applies?

Statistical: what distribution is expected?

Privacy: what restrictions apply?

AI use: which systems may consume it?



Producer → Contract → Automated validation.

PASS → Data Product.

FAIL → Quarantine.

Implement contracts as machine-readable and automatically enforced controls rather than documents stored in a repository.



Traditional classification: Public, Internal, Confidential, Restricted.
Extend it with AI-use classification: Approved for AI, Restricted for AI and Prohibited for AI.
A dataset can be accessible internally but prohibited for model training.



Traditional ML often consumes structured features.

GenAI consumes documents, knowledge and context.

New requirements include document permissions, chunk-level access, source citation, document version, retrieval permissions, confidentiality and prompt or data leakage controls.



Detect → Classify → Identify impacted datasets → Identify impacted AI consumers → Contain → Correct → Reprocess → Postmortem.
Implement incident ownership, escalation, runbooks and recovery procedures.



Example: a CRM field changes → Customer Data Product → 12 downstream datasets → 7 AI applications → 3 automated decisions. Implement impact analysis through lineage so source changes can be traced to affected consumers.



Business Data Owner: meaning and business accountability.

Data Steward: quality and definitions.

Data Engineer: pipelines.

Data Platform Team: infrastructure.

AI/Data Scientist: consumption requirements.

Risk/Compliance: controls.



Assess every critical dataset for owner, business definition, quality, lineage, versioning, point-in-time history, data contract, AI-use classification, drift monitoring and reproducibility.

Create an AI Data Readiness Score and prioritize gaps based on business impact.

90-Day Transformation Roadmap



Days 0–30: identify priority AI use cases, critical data, owners, sources, quality and gaps.

Days 31–60: implement data contracts, data products, lineage, versioning, quality gates and AI classification.

Days 61–90: implement point-in-time datasets, observability, drift monitoring, AI access policies, reproducibility and an AI data dashboard.

What the Enterprise Architecture Should Look Like

innovate

achieve

lead

AI, ML and GenAI sit above an AI Data Access layer.

Below are governed Data Products supported by Quality, Metadata and Governance.

These sit on the Data Platform and Integration Layer, which connect ERP, CRM, IoT, documents and external sources.

Key message:

AI should consume governed enterprise data products, not uncontrolled copies of enterprise data.



Traditional EDM: data is managed, governed, integrated and quality-controlled.

AI-ready EDM: data is fit for purpose, temporally correct, reproducible, traceable to decisions, purpose-controlled, continuously observable and delivered as governed products.

AI-ready data management is not a replacement for Enterprise Data Management. It is its evolution from managing data as an organizational asset to managing data as a continuously changing input to machine-driven decisions.